Introducción a los modelos de Record Linkage

Andrés Gutiérrez - División de Estadísticas de la CEPAL

En este repositorio me propongo desarrollar, de manera detallada y progresiva, tanto la teoría como algunos elementos prácticos del *record linkage*, tomando como eje tres hitos fundamentales en la literatura:

1. El modelo clásico de comparación de pares de Fellegi-Sunter (1969)
2. El modelo de *matching* con errores de medición en las covariables (*Hit-Miss*) de Copas y Hilton (1990)
3. El modelo de *matching* bipartito propuesto por Sadinle (2014)

Más que un manual exhaustivo o una referencia definitiva, este repositorio nace como un ejercicio personal: un intento de organizar, comprender y explicar estos modelos desde sus fundamentos estadísticos y matemáticos, motivado principalmente por mi curiosidad intelectual. En ese sentido, el énfasis está puesto en descomponer las ideas, hacer explícitas las suposiciones y recorrer paso a paso las estructuras probabilísticas que sustentan cada enfoque.

El hilo conductor de esta exploración es la inferencia bayesiana, que permite unificar de manera natural los distintos modelos bajo un mismo marco conceptual. Por esta razón, además del desarrollo teórico, incluyo implementaciones computacionales que buscan reflejar fielmente las formulaciones probabilísticas. Dichas implementaciones están escritas en STAN, no sólo como herramienta de cálculo, sino también como un lenguaje que ayuda a traducir la teoría en modelos explícitos y reproducibles.

En síntesis, este repositorio debe entenderse como una especie de cuaderno de trabajo extendido: un espacio donde la teoría, la modelación y la implementación dialogan constantemente, con el objetivo principal de profundizar en la comprensión del *record linkage* desde una perspectiva rigurosa pero también exploratoria.

El modelo de Fellegi-Sunter

El **record linkage** es el proceso de identificar registros que corresponden a la misma entidad (por ejemplo, la misma persona) en diferentes bases de datos. Su objetivo es determinar si un par de registros corresponde a:

- **Match**: la misma entidad
- **No match**: entidades diferentes

El enfoque probabilístico modela esta decisión utilizando probabilidades en lugar de reglas determinísticas. Para cada par construimos un **vector de comparaciones**:

$$X_i = (X_{i1}, X_{i2}, ..., X_{iK})$$

En donde, K representa el número de variables que se utilizan en la comparación, X_{ik} es un indicador de coincidencia en la variable k, definido por:

$$X_{ik} = \begin{cases} 1 & \text{si hay coincidencia} \\ 0 & \text{si no coincide} \end{cases}$$

Por ejemplo, si se consideran tres variables de comparación (nombre, apellido y fecha de nacimiento) y $X_i = (1, 1, 0)$, es porque, para ese par específico hay coincidencia en las primeras dos variables (nombre y apellido), pero no en la última (fecha de nacimiento).

En las aplicaciones prácticas nunca sabremos si una entidad en una lista realmente coincide con otra entidad en otra lista, por eso es necesario crear una variable latente que pueda modelar este proceso de identificación. De esta forma, para cada par, introducimos la variable no observable, Z_i, definida así:

$$Z_i = \begin{cases} 1 & \text{si el par es match} \\ 0 & \text{si el par no es match} \end{cases}$$

Por tanto, nuestro objetivo será calcular la probabilidad de que un par de registros corresponda a la misma entidad (match) $\pi = P(Z = 1)$. Como estamos involucrando un conjunto de variables de comparación, entonces para cada par debemos calcular la probabilidad condicionada a X_i, así:

$$P(Z_i = 1|X_i)$$

Esto es, la probabilidad de que dos registros correspondan a la misma persona dado su patrón de coincidencias. En muchos contextos se espera que esta probabilidad sea pequeña, porque la mayoría de los pares posibles no corresponderán al mismo registro.

En el modelo probabilístico de record linkage, cada variable de comparación puede tener un comportamiento distinto en términos de su capacidad para coincidir cuando dos registros pertenecen realmente a la misma entidad. Por esta razón, el modelo introduce un parámetro específico para cada variable:

$$m_k = P(X_k = 1|Z = 1)$$

Estos parámetros representan la probabilidad de coincidencia cuando el par sí es match. La razón por la cual cada variable tiene asociada su propio parámetro es que no todas las variables son igual de confiables ni presentan los mismos niveles de error o variabilidad en las bases de datos. Por ejemplo:

1. El nombre puede coincidir en la mayoría de los casos cuando se trata de la misma persona, pero puede haber discrepancias debido a errores de digitación. Por esta razón, podríamos tener una probabilidad como $m_{nombre} = 0.9$. Esto significa que incluso cuando los registros corresponden a la misma persona, existe un 10% de probabilidad de que los nombres no coincidan exactamente.
2. Los apellidos suelen ser más estables que los nombres, aunque todavía pueden ocurrir discrepancias debido a errores de digitación, o diferencias en acentuación ("Pérez" vs "Perez"). Debido a esta mayor estabilidad, podría ser razonable que $m_{apellido} = 0.9$. Lo que indica que solo en un 5% de los casos habría discrepancia incluso cuando se trata de la misma persona.
3. La fecha de nacimiento suele ser una variable altamente informativa y relativamente estable. Cuando los registros corresponden a la misma persona, lo normal es que coincida exactamente. Sin embargo, pueden existir errores debido a formatos distintos de fecha ("03/11/1988" vs "11/03/1988"). Si estos errores son menos frecuentes, podríamos tener $m_{fecha} = 0.98$. Lo que indicaría que solo en un 2% de los casos habría discrepancia incluso cuando los registros son realmente del mismo individuo.

Variable	m_k
nombre	0.90
apellido	0.95
fecha nacimiento	0.98

Ahora, asumiendo independencia entre variables, se tiene que:

$$P(X|Z=1) = \prod_{k=1}^{K} m_k^{X_k}(1-m_k)^{1-X_k}$$

De manera análoga, el modelo de record linkage introduce otro conjunto de probabilidades que describen el comportamiento de las variables cuando los registros pertenecen a personas distintas. Estas probabilidades se definen como:

$$u_k = P(X_k = 1|Z = 0)$$

Los cuales son probabilidades que representan coincidencias por azar, dado que el par no corresponde a la misma entidad. Por ejemplo:

1. Incluso cuando dos registros corresponden a personas diferentes, puede ocurrir que tengan el mismo nombre. Esto sucede porque muchos nombres son relativamente comunes. En este contexto, podríamos tener $u_{nombre} = 0.05$. Esto significa que aproximadamente un 5% de los pares de personas distintas podrían coincidir en el nombre. Aunque si la base de datos incluye nombres muy comunes, este valor de podría ser relativamente alto.
2. Los apellidos suelen ser más informativos que los nombres, especialmente en poblaciones donde la diversidad de apellidos es mayor. Sin embargo, todavía pueden existir coincidencias por azar (múltiples personas con apellido "García"); por tanto podríamos tener $u_{apellido} = 0.02$. Esto indica que alrededor del 2% de los pares de personas distintas coincidirían en el apellido.
3. La fecha de nacimiento suele ser una variable muy discriminante. La probabilidad de que dos personas distintas compartan exactamente la misma fecha de nacimiento es relativamente baja, especialmente si se considera día, mes y año completos. Por ejemplo, podríamos tener $u_{fecha} = 0.01$. Esto significa que solo en aproximadamente el 1% de los pares de personas diferentes se observaría coincidencia exacta en la fecha de nacimiento.

Variable	u_k
nombre	0.05
apellido	0.02
fecha nacimiento	0.01

Una vez más, asumiendo independencia entre variables, se tiene que:

$$P(X|Z=0) = \prod_{k=1}^{K} u_k^{X_k}(1-u_k)^{1-X_k}$$

La razón de verosimilitudes como regla decisión

Desde un punto de vista frecuentista, el estadístico central del modelo de Fellegi-Sunter es la razón de verosimilitudes, dada por la siguiente expresión

$$\Lambda(X) = \frac{P(X \mid Z=1)}{P(X \mid Z=0)} = \frac{\prod_{k=1}^{K} m_k^{X_k}(1-m_k)^{1-X_k}}{\prod_{k=1}^{K} u_k^{X_k}(1-u_k)^{1-X_k}}$$

Es posible demostrar que la regla de decisión óptima consiste en clasificar los pares utilizando este estadístico. Nótese que cuando se comparan varias variables, es posible que las productorias en $P(X \mid Z=1)$ y $P(X \mid Z=0)$ se acerquen a cero, y que se tengan problemas de numéricos de convergencia computacional (underflow).

Por esta razón, es usual trabajar en escala logarítmica, Si sustituimos las expresiones anteriores, se obtiene nuevamente un producto de términos asociados a cada variable de comparación. Sin embargo, resulta más conveniente trabajar en escala logarítmica, ya que el logaritmo transforma las productorias en sumatorias, evitando problemas de magnitud computacional. En efecto, acudiendo a las reglas del logaritmo, y después de un poco de álgebra, se tiene que

$$\log \Lambda(X) = \sum_{k=1}^{K} \left[X_k \log\left(\frac{m_k}{u_k}\right) + (1 - X_k) \log\left(\frac{1 - m_k}{1 - u_k}\right) \right].$$

De esta forma, la evidencia aportada por cada variable puede interpretarse como un peso, y el estadístico total se obtiene simplemente sumando los pesos asociados a los acuerdos y desacuerdos observados. En particular, para cada variable k se definen los pesos:

$$\begin{aligned} w_k^{(Z=1)} &= \log\left(\frac{m_k}{u_k}\right) \\ w_k^{(Z=0)} &= \log\left(\frac{1 - m_k}{1 - u_k}\right) \end{aligned}$$

De esta manera, cada variable de comparación aporta una cierta cantidad de evidencia a favor o en contra de que el par de registros corresponda a la misma entidad. Esta evidencia se resume mediante los pesos definidos anteriormente. Al combinar la información proveniente de todas las variables consideradas, se obtiene un peso total que sintetiza la evidencia global del par, el cual se expresa como

$$W(X) = \sum_{k=1}^{K} w_k.$$

Este estadístico resume la información contenida en el vector de comparaciones X y permite ordenar los pares de registros de acuerdo con la evidencia de que correspondan a la misma entidad. Valores altos de $W(X)$ indican una mayor probabilidad de que el par sea un match, mientras que valores bajos sugieren que se trata de registros correspondientes a entidades distintas.

A partir de este estadístico, el modelo de Fellegi–Sunter propone una regla de decisión basada en dos umbrales. En particular, se definen dos valores T_U y T_M que permiten clasificar los pares de registros en tres posibles categorías: matches, non-matches o pares que requieren revisión manual. Formalmente, la regla de decisión se expresa como

$$\begin{cases} W(X) \geq T_M & \text{clasificar como match} \\ W(X) \leq T_U & \text{clasificar como non-match} \\ T_U < W(X) < T_M & \text{revisión clerical} \end{cases}$$

La presencia de una región intermedia responde al hecho de que, en algunos casos, la evidencia disponible no es suficientemente concluyente para tomar una decisión automática. En esas situaciones, el modelo recomienda que los pares sean evaluados mediante una revisión manual. Finalmente, los valores de los umbrales se determinan de modo que se controlen explícitamente las probabilidades de error del procedimiento de clasificación. En particular, el modelo considera dos tipos de error:

- **Error tipo I:** clasificar un *non-match* como *match*.
- **Error tipo II:** clasificar un *match* como *non-match*.

La elección de los umbrales T_U y T_M permite equilibrar estos errores y establecer un compromiso entre la tasa de clasificaciones automáticas y la necesidad de revisión manual de los pares ambiguos.

Probabilidad posterior de emparejamiento

El objetivo del modelo es calcular la probabilidad de que el par sea un match dado el patrón observado de coincidencias en la variables de comparación. Esto se expresa como

$$P(Z = 1 \mid X)$$

Y se conoce como la probabilidad posterior de match. Su interpretación es sencilla: dado el patrón de coincidencias entre los registros, $P(Z = 1 \mid X)$ representa la probabilidad de que correspondan a la misma persona. Para calcular esta probabilidad es necesario recurrir a la regla de Bayes, la cual establece que:

$$P(Z = 1|X) = \frac{P(X|Z = 1)P(Z = 1)}{P(X)}$$

En donde $P(Z = 1)$ es la probabilidad previa de emparejamiento, que representa la probabilidad de que un par de registros sea un match antes de observar las variables de comparación. Esta cantidad suele ser pequeña porque la mayoría de los pares posibles corresponden a personas diferentes. Además, $P(X)$, la probabilidad marginal del patrón, representa la probabilidad total de observar el patrón de coincidencias X, que, acudiendo al teorema de probabilidad total, se escribe de la siguiente manera:

$$P(X) = P(X|Z = 1)P(Z = 1) + P(X|Z = 0)P(Z = 0)$$

En donde

$$P(Z = 0) = 1 - P(Z = 1)$$

Por ende, notando que $\pi = P(Z = 1)$, y sustituyendo la expresión de $P(X)$ en el teorema de Bayes, se obtiene:

$$P(Z = 1|X) = \frac{\pi P(X|Z = 1)}{\pi P(X|Z = 1) + (1 - \pi)P(X|Z = 0)}$$

Si el patrón observado es mucho más probable bajo el modelo de match, entonces $P(Z = 1|X)$ será cercano a 1; si el patrón es más probable bajo el modelo de no-match, la probabilidad posterior será cercana a 0. Este cálculo permite transformar el patrón de coincidencias entre registros en una probabilidad cuantitativa de que el par sea un match y corresponde a la base del modelo probabilístico de **Fellegi–Sunter** y de muchos algoritmos modernos de record linkage.

Ejemplo

Supongamos que tenemos dos listas, como se describe a continuación:

idA	nombre	apellido	fecha_nac
A1	Juan	Pérez	1990-05-10
A2	María	Gómez	1988-11-03

idB	nombre	apellido	fecha_nac
B1	Juan	Pérez	1990-05-10

idB	nombre	apellido	fecha_nac
B2	María	Gómez	1987-11-03
B3	Carlos	López	1992-07-21

El primer paso es construir **todos los pares posibles** entre las dos listas. Como la lista A tiene 2 registros y la lista B tiene 3 registros, entonces el número total de pares es $2 \times 3 = 6$ y la tabla de todos los posibles pares es:

idA	idB	nombre_A	nombre_B	apellido_A	apellido_B	fnac_A	fnac_B
A1	B1	Juan	Juan	Pérez	Pérez	1990-05-10	1990-05-10
A1	B2	Juan	María	Pérez	Gómez	1990-05-10	1987-11-03
A1	B3	Juan	Carlos	Pérez	López	1990-05-10	1992-07-21
A2	B1	María	Juan	Gómez	Pérez	1988-11-03	1990-05-10
A2	B2	María	María	Gómez	Gómez	1988-11-03	1987-11-03
A2	B3	María	Carlos	Gómez	López	1988-11-03	1992-07-21

En una segunda instancia, transformamos las variables originales en variables de comparación binarias para crear la matriz X, así:

idA	idB	nombre_match	apellido_match	fnac_match
A1	B1	1	1	1
A1	B2	0	0	0
A1	B3	0	0	0
A2	B1	0	0	0
A2	B2	1	1	0
A2	B3	0	0	0

De esta forma, cada par tiene un vector de comparación

$$X_i = (x_{i1}, x_{i2}, x_{i3})$$

Además, supongamos que la probabilidad a priori de que un par de registros corresponda a la misma entidad es $\pi = 0.2$ y que se cuenta con los siguientes valores para los parámetros del modelo de emparejamiento, que representan las probabilidades de acuerdo entre variables cuando los registros sí corresponden a la misma entidad (m_k) y cuando no corresponden a la misma entidad (u_k)::

Variable	m_k	u_k
nombre	0.9	0.1
apellido	0.9	0.1
fecha nacimiento	0.8	0.05

Consideremos ahora el par A2–B2. Para este par se observa el vector de comparaciones $X_2 = (1, 1, 0)$, lo que significa que coinciden el nombre y el apellido, pero no coincide la fecha de nacimiento. A partir de los parámetros del modelo, es posible calcular la probabilidad de observar ese patrón de coincidencias bajo cada uno de los dos posibles estados: que los registros correspondan a la misma entidad ($Z = 1$) o que correspondan a entidades distintas ($Z = 0$).

Asimismo, los pesos de Fellegi–Sunter son los siguientes:

Variable	m_k	u_k	Peso acuerdo $w_k^{(1)}$	Peso desacuerdo $w_k^{(0)}$
nombre	0.9	0.1	$\log(9) = 2.197$	$\log(0.111) = -2.197$
apellido	0.9	0.1	$\log(9) = 2.197$	$\log(0.111) = -2.197$
fecha nacimiento	0.8	0.05	$\log(16) = 2.773$	$\log(0.2105) = -1.558$

Por ende, para el par A2–B2, cuyo vector de comparación es $X_2 = (1, 1, 0)$, el peso total sería

$$W(X_2) = 2.197 + 2.197 - 1.558 = 2.836$$

Y para todos los pares, tendríamos la siguiente tabla con su interpretación.

Par	Vector X	Peso $W(X)$	Score final	Interpretación
A1–B1	(1,1,1)	7.167	5.781	match casi seguro
A1–B2	(0,0,0)	-5.952	-7.338	no match
A1–B3	(0,0,0)	-5.952	-7.338	no match
A2–B1	(0,0,0)	-5.952	-7.338	no match
A2–B2	(1,1,0)	**2.836**	**1.450**	match probable
A2–B3	(0,0,0)	-5.952	-7.338	no match

Ahora, desde un punto de vista probabilístico, también se obtienen las mismas conslusiones. En efecto, para el par A2-B2, se obtiene que:

- $P(X_2|Z = 1) = m_1 \times m_2 \times (1 - m_3) = 0.9 \times 0.9 \times 0.2 = 0.162$
- $P(X_2|Z = 0) = u_1 \times u_2 \times (1 - u_3) = 0.1 \times 0.1 \times 0.95 = 0.0095$.

Una vez calculadas estas probabilidades, se puede aplicar la ecuación de la probabilidad posterior de match, que combina la evidencia observada en el vector de comparación con la probabilidad a priori π:

$$P(Z = 1 \mid X_2) = \frac{\pi P(X_2 \mid Z = 1)}{\pi P(X_2 \mid Z = 1) + (1 - \pi)P(X_2 \mid Z = 0)} = \frac{0.2 \times 0.162}{0.2 \times 0.162 + 0.8 \times 0.0095} = 0.81$$

Es decir, la probabilidad de que este par de registros corresponda a la misma entidad es 0.81, lo que sugiere que se trata de un emparejamiento bastante plausible. De manera análoga, si se realiza este mismo cálculo para todos los posibles pares de registros entre las dos bases de datos, se obtiene la siguiente tabla de probabilidades de match:

Par	Vector X	Probabilidad de match
A1–B1	(1,1,1)	**0.997**
A1–B2	(0,0,0)	0.00065
A1–B3	(0,0,0)	0.00065
A2–B1	(0,0,0)	0.00065
A2–B2	(1,1,0)	**0.81**
A2–B3	(0,0,0)	0.00065

A partir de estos resultados, se observa que el modelo está identificando dos pares con probabilidades sustanciales de corresponder a la misma entidad: el par A1–B1 (match casi seguro) y el par A2–B2 (match

probable). En contraste, los demás pares presentan probabilidades muy bajas, lo que indica que es muy poco probable que correspondan a la misma persona.

En la práctica, los algoritmos modernos aplican exactamente esta misma lógica, pero sobre millones de pares de registros, utilizando múltiples variables de comparación y técnicas computacionales que permiten realizar estos cálculos de manera eficiente.

Inferencia bayesiana

La inferencia bayesiana constituye un marco general para realizar inferencia estadística a partir de datos observados y conocimiento previo sobre los parámetros de interés. El punto de partida es la especificación de la verosimilitud del modelo, es decir, la distribución de probabilidad que describe cómo se generan los datos en función de un conjunto de parámetros desconocidos. Esta verosimilitud resume el mecanismo probabilístico que conecta el modelo con la información empírica disponible y representa la contribución de los datos al proceso de inferencia.

Una vez definida la verosimilitud, se deben establecer distribuciones previas para los parámetros del modelo, las cuales reflejan el conocimiento o las creencias existentes antes de observar los datos. La combinación de la verosimilitud con estas distribuciones previas, mediante el teorema de Bayes, permite obtener la distribución posterior, que representa la distribución de probabilidad de los parámetros condicionada a los datos observados. En muchos modelos de interés la distribución posterior no puede obtenerse de forma analítica, por lo que se recurre a métodos computacionales, como los algoritmos de simulación tipo Markov Chain Monte Carlo (MCMC), para aproximarla y realizar inferencias sobre los parámetros del modelo.

Verosimilitud del modelo

Sea Z_i_la variable latente que indica si el par i corresponde a un *match* o a un *non-match*. En el modelo se asume que esta variable sigue una distribución Bernoulli con probabilidad π de que el par corresponda a la misma persona:

$$Z_i \sim \text{Bernoulli}(\pi)$$

Condicionalmente al estado de Z_i, se modela el comportamiento de las variables de comparación. En particular, cada componente del vector de comparaciones se modela mediante una distribución Bernoulli. Para el caso en que el par corresponde a la misma persona ($Z_i = 1$), se tiene que

$$X_{ik}|Z = 1 \sim Bernoulli(m_k)$$

donde el parámetro $m_k = P(X_k = 1|Z = 1)$ representa la probabilidad de acuerdo en el campo k cuando los registros corresponden al mismo individuo. De forma análoga, cuando los registros no corresponden a la misma persona ($Z_i = 0$), se define un modelo similar pero con un parámetro diferente que captura la probabilidad de acuerdo entre registros distintos:

$$X_{ik}|Z = 0 \sim Bernoulli(u_k)$$

donde $u_k = P(X_k = 1|Z = 0)$. Este parámetro representa la probabilidad de acuerdo en el campo k cuando el par corresponde a registros de personas diferentes.

Bajo el supuesto de independencia condicional entre las variables de comparación dado el estado de Z_i, la distribución conjunta del vector de comparaciones puede expresarse como el producto de las distribuciones de cada componente. En consecuencia, las distribuciones condicionales del vector X_i vienen dadas por

$$P(X_i \mid Z_i = 1) = \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 1) = \prod_{k=1}^{K} m_k^{X_{ik}} (1 - m_k)^{1 - X_{ik}}$$

$$P(X_i \mid Z_i = 0) = \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 0) = \prod_{k=1}^{K} u_k^{X_{ik}} (1 - u_k)^{1 - X_{ik}}.$$

A partir de estas distribuciones condicionales es posible obtener la distribución marginal del vector de comparaciones. Para ello se aplica el teorema de probabilidad total, considerando los dos posibles estados de la variable latente Z_i. De esta forma, la distribución marginal de X_i se expresa como

$$\begin{aligned} P(X_i) &= \pi P(X_i \mid Z_i = 1) + (1 - \pi) P(X_i \mid Z_i = 0) \\ &= \pi \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 1) + (1 - \pi) \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 0) \end{aligned}$$

Si se observan N pares de registros independientes, la verosimilitud del modelo se obtiene como el producto de las probabilidades marginales de cada vector de comparaciones observado. En consecuencia, la función de verosimilitud puede escribirse como

$$\begin{aligned} P(X|m, u, \pi) &= \prod_{i=1}^{N} P(X_i) \\ &= \prod_{i=1}^{N} \left[\pi \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 1) + (1 - \pi) \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 0) \right] \end{aligned}$$

Para facilitar la estimación del modelo, resulta conveniente trabajar en escala logarítmica, ya que esto permite transformar productos en sumas y simplificar los cálculos. Aplicando propiedades matemáticas, se tiene que

$$\log P(X|m, u, \pi) = \sum_{i=1}^{N} \log \left[\pi \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 1) + (1 - \pi) \prod_{k=1}^{K} P(X_{ik} \mid Z_i = 0) \right]$$

$$\begin{aligned} &= \sum_{i=1}^{N} \log \Bigg[\exp \left(\log \pi + \sum_{k=1}^{K} \log P(X_{ik} \mid Z_i = 1) \right) \\ &\qquad + \exp \left(\log(1 - \pi) + \sum_{k=1}^{K} \log P(X_{ik} \mid Z_i = 0) \right) \Bigg] \end{aligned}$$

Esta última expresión corresponde precisamente al tipo de funciones que se utilizan en la implementación computacional del modelo. En particular, la estructura $\log [\exp(\cdot) + \exp(\cdot)]$ es evaluada mediante transformaciones numéricamente estables. Este tipo de cálculos es el que emplea el software de inferencia bayesiana STAN, el cual implementa funciones especializadas para evaluar expresiones de este tipo evitando problemas de precisión numérica y permitiendo estimar eficientemente los parámetros del modelo.

Distribuciones previas

En un enfoque completamente bayesiano, la incertidumbre acerca de la localización de los parámetros que gobiernan el modelo probabilístico se trata como una variable aleatoria y por tanto se asignan distribuciones previas a cada uno de ellos. En el modelo de Fellegi–Sunter, los parámetros principales son m_k, la probabilidad de acuerdo entre registros verdaderamente coincidentes; u_k, la probabilidad de acuerdo entre registros que no corresponden a la misma entidad; π, la proporción de pares que corresponden a verdaderas coincidencias en el conjunto de comparaciones.

Una elección natural para estos parámetros son distribuciones Beta, ya que se trata de probabilidades definidas en $[0, 1]$. De esta forma:

$$m_k \sim Beta(a_m, b_m)$$

$$u_k \sim Beta(a_u, b_u)$$

$$\pi \sim Beta(a_\pi, b_\pi)$$

Estas distribuciones previas permiten incorporar conocimiento previo sobre la calidad de las variables de comparación, estimar la incertidumbre en los parámetros y obtener la distribución posterior completa mediante métodos computacionales como MCMC. La siguiente tabla muestra algunas distribuciones previas que podrían usarse, dependiendo de la situación en la que estemos.

Parámetro	Distribución sugerida	Interpretación
m_k	$Beta(9, 1)$	Variable altamente discriminante, alta probabilidad de acuerdo en verdaderos matches.
m_k	$Beta(5, 2)$	Variable moderadamente confiable, algo de incertidumbre en la coincidencia.
u_k	$Beta(1, 9)$	Probabilidad baja de acuerdo en pares no coincidentes.
u_k	$Beta(2, 8)$	Variable con mayor posibilidad de coincidencias aleatorias.
π	$Beta(2, 8)$	Baja proporción de verdaderos matches en el conjunto de pares.
π	$Beta(5, 5)$	Prior poco informativa, refleja incertidumbre sobre la proporción de matches.

Distribución Posterior

Una vez definidas la verosimilitud del modelo y las distribuciones previas de los parámetros, la inferencia bayesiana se centra en la distribución posterior, que combina la información de los datos con el conocimiento previo. Formalmente, al asumir independencia entre los parámetros, la distribución posterior se obtiene aplicando la regla de Bayes, de la siguiente forma:

$$P(m, u, \pi \mid X) \propto P(X \mid m, u, \pi) \cdot P(m) \cdot P(u) \cdot P(\pi)$$

donde $P(X \mid m, u, \pi)$ es la verosimilitud de los datos observados y $P(m)$, $P(u)$, $P(\pi)$ son las distribuciones previas de los parámetros. La distribución posterior refleja nuestra incertidumbre actualizada sobre los parámetros del modelo después de observar los datos. A diferencia de un estimador puntual, esta distribución posterior proporciona un marco completo para cuantificar la incertidumbre, permitiendo calcular intervalos de credibilidad, probabilidades de que los parámetros superen ciertos umbrales, o directamente las probabilidades de que cada par de registros sea un match verdadero.

En la práctica, la posterior raramente se puede obtener de manera analítica debido a la complejidad del modelo y la multiplicidad de variables de comparación. Por ello, se recurre a métodos computacionales, como algoritmos de simulación tipo Markov Chain Monte Carlo (MCMC), para generar muestras de la posterior. Estas muestras permiten realizar inferencia sobre los parámetros y sobre los pares de registros, calcular probabilidades posteriores de coincidencia, y tomar decisiones basadas en un marco probabilístico sólido y coherente.

Implementación paso a paso en `RecordLinkage.r`

En este capítulo descompongo el archivo `FellegiSunter.r` en bloques lógicos para mostrar qué hace cada parte del código y por qué es necesaria dentro de un flujo simple de *record linkage* bayesiano.

Carga de librerías y configuración inicial

El script comienza limpiando el entorno de trabajo, cargando las librerías necesarias y definiendo algunas opciones para que STAN pueda compilar y ejecutar el modelo de manera más eficiente.

```r
############################################################
# 1. Librerías
############################################################

rm(list = ls())

library(rstan)
library(fastLink)

rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())

set.seed(123)
```

El código comienza limpiando el entorno de trabajo mediante la función `rm(list = ls())`, lo que elimina todos los objetos existentes en memoria y evita que resultados previos interfieran con la ejecución actual. A continuación, se cargan las librerías necesarias: `library(rstan)` permite utilizar STAN desde R para ajustar modelos bayesianos, mientras que `library(fastLink)` incorpora una implementación ya existente de record linkage, lo que resulta útil para comparar su desempeño con el modelo que se desarrollará en STAN.

Posteriormente, se configuran algunas opciones clave para mejorar la eficiencia y reproducibilidad del análisis. La instrucción `rstan_options(auto_write = TRUE)` permite reutilizar archivos compilados por STAN, reduciendo significativamente los tiempos de ejecución en corridas posteriores. Por su parte, `options(mc.cores = parallel::detectCores())` habilita el uso de todos los núcleos disponibles del procesador, lo que facilita la ejecución paralela de múltiples cadenas MCMC. Finalmente, `set.seed(123)` fija la semilla aleatoria, garantizando que los resultados obtenidos puedan reproducirse exactamente en futuras ejecuciones.

Construcción de los listados que serán emparejados

Luego se generan dos listas de registros. La base `A` representa un primer archivo y la base `B` un segundo archivo. Una parte de `B` se construye para que contenga coincidencias reales con `A`, mientras que otra parte agrega registros extra que no necesariamente tienen correspondencia. Es decir, se construye un conjunto de datos sintético que simula dos bases de información a ser vinculadas. Primero se define el tamaño de ambas listas, donde `A` contiene 50 registros y `B` contiene 60. Luego se crean catálogos simples de nombres, apellidos y sexo, los cuales sirven como insumos para generar variables de identificación. A partir de estos catálogos, se construye la base `A` como un conjunto de datos sintético que incluye un identificador único, nombre, apellido, edad y sexo para cada individuo.

A continuación, se introduce un mecanismo controlado de coincidencias reales entre ambas bases. Se define que los primeros 30 registros de `B` corresponden verdaderamente a los primeros 30 registros de `A`. Para ello:

1. Se construye `B_match` copiando esos registros, pero incorporando una pequeña perturbación en la variable edad de -1, 0 o 1, con el fin de imitar errores o variaciones típicas entre fuentes de datos.
2. Luego se genera `B_extra` con registros adicionales independientes que no tienen correspondencia en `A`, y finalmente se concatenan ambas partes para formar la base final `B`.

En esencia, este procedimiento crea un escenario controlado en el que se conoce la verdad subyacente, lo que resulta especialmente útil para evaluar si el método de record linkage es capaz de recuperar correctamente las coincidencias reales.

```r
##############################################################
# 2. Crear listas A y B
##############################################################

nA <- 50
nB <- 60

nombres <- c("Juan","Maria","Pedro","Ana","Luis","Carlos",
             "Lucia","Jorge","Sofia","Miguel","Laura")

apellidos <- c("Gomez","Perez","Rodriguez","Lopez","Garcia")

sexos <- c("M","F")

A <- data.frame(
  idA = 1:nA,
  nombre = sample(nombres,nA,replace=TRUE),
  apellido = sample(apellidos,nA,replace=TRUE),
  edad = sample(20:60,nA,replace=TRUE),
  sexo = sample(sexos,nA,replace=TRUE)
)

n_match_true <- 30

B_match <- data.frame(
  idB = 1:n_match_true,
  nombre = A$nombre[1:n_match_true],
  apellido = A$apellido[1:n_match_true],
  edad = A$edad[1:n_match_true] + sample(c(-1,0,1),n_match_true,replace=TRUE),
  sexo = A$sexo[1:n_match_true]
)
```

```r
B_extra <- data.frame(
  idB = (n_match_true+1):nB,
  nombre = sample(nombres,nB-n_match_true,replace=TRUE),
  apellido = sample(apellidos,nB-n_match_true,replace=TRUE),
  edad = sample(20:60,nB-n_match_true,replace=TRUE),
  sexo = sample(sexos,nB-n_match_true,replace=TRUE)
)

B <- rbind(B_match,B_extra)
```

A continuación se muestran los primeros seis registros de ambos listados.

```
##   idA nombre apellido edad sexo
## 1   1  Pedro    Gomez   38    M
## 2   2  Pedro   Garcia   44    F
## 3   3 Miguel    Gomez   58    M
## 4   4  Maria    Perez   45    F
## 5   5 Carlos    Lopez   28    M
## 6   6  Laura    Lopez   26    M
```

```
##   idB nombre apellido edad sexo
## 1   1  Pedro    Gomez   37    M
## 2   2  Pedro   Garcia   44    F
## 3   3 Miguel    Gomez   58    M
## 4   4  Maria    Perez   45    F
## 5   5 Carlos    Lopez   29    M
## 6   6  Laura    Lopez   25    M
```

3. Generación de todos los pares posibles

Una vez construidas las dos bases, el siguiente paso es formar el producto cartesiano entre `A` y `B`, es decir, todos los pares candidato que podrían compararse.

```r
##############################################################
# 3. Crear todos los pares
##############################################################

pairs <- merge(A,B,by=NULL)

N <- nrow(pairs)
```

¿Qué hace este bloque?

- `merge(A, B, by = NULL)` genera todos los pares posibles entre registros de `A` y registros de `B`.
- Si `A` tiene 50 filas y `B` tiene 60, entonces el número total de pares es `50 x 60 = 3000`.
- `N <- nrow(pairs)` guarda ese total, porque será una de las entradas del modelo en STAN.

Este paso transforma el problema de enlazar dos archivos en un problema de clasificación de pares: cada par debe ser considerado como posible *match* o *non-match*.

4. Construcción de variables de comparación

Después de formar todos los pares, el código compara campo por campo para resumir el nivel de acuerdo entre los registros. Estas comparaciones son justamente las variables que alimentan el modelo de Fellegi-Sunter.

```
#############################################################
# 4. Variables de comparación
#############################################################

pairs$nombre_match <- ifelse(pairs$nombre.x == pairs$nombre.y,1,0)
pairs$apellido_match <- ifelse(pairs$apellido.x == pairs$apellido.y,1,0)
pairs$edad_match <- ifelse(abs(pairs$edad.x - pairs$edad.y)<=1,1,0)
pairs$sexo_match <- ifelse(pairs$sexo.x == pairs$sexo.y,1,0)

X <- as.matrix(pairs[,c(
  "nombre_match",
  "apellido_match",
  "edad_match",
  "sexo_match"
)])

K <- ncol(X)
```

¿Qué hace este bloque?

- Crea cuatro indicadores binarios de acuerdo:
  - `nombre_match`: vale 1 si los nombres coinciden exactamente.
  - `apellido_match`: vale 1 si los apellidos coinciden exactamente.
  - `edad_match`: vale 1 si las edades difieren a lo sumo en una unidad.
  - `sexo_match`: vale 1 si el sexo coincide.
- Reúne esas variables en una matriz `X`, donde:
  - cada fila representa un par de registros,
  - cada columna representa una variable de comparación.
- Guarda en `K` el número de variables de comparación, que aquí es 4.

Esta matriz `X` es el insumo central del modelo bayesiano: cada fila resume la evidencia observada a favor o en contra de que un par sea una coincidencia real.

5. Definición de la verdad simulada

Como este ejemplo es artificial, el script puede construir una variable que identifica cuáles pares son verdaderamente coincidencias.

```
#############################################################
# 5. Verdad simulada
#############################################################

pairs$true_match <- ifelse(
  pairs$idA <= n_match_true &
    pairs$idB == pairs$idA,
  1,0)
```

¿Qué hace este bloque?

- Marca con 1 los pares en los que:
  - `idA` está entre los primeros 30 registros con coincidencia real, y
  - `idB == idA`, que es la forma específica en que se construyó la correspondencia entre `A` y `B_match`.
- Todos los demás pares quedan marcados como 0.

Esta variable no se usa para ajustar el modelo bayesiano. Su función es servir como referencia para evaluar el desempeño del método una vez que se haya estimado.

6. Aplicación de `fastLink`

Antes de ajustar el modelo en STAN, el script ejecuta el paquete `fastLink` como punto de comparación.

```
############################################################
# 6. fastLink
############################################################

A$edad <- as.numeric(A$edad)
B$edad <- as.numeric(B$edad)

A$nombre <- as.character(A$nombre)
B$nombre <- as.character(B$nombre)

A$apellido <- as.character(A$apellido)
B$apellido <- as.character(B$apellido)

A$sexo <- as.character(A$sexo)
B$sexo <- as.character(B$sexo)

fl <- fastLink(
  dfA = A,
  dfB = B,
  varnames = c("nombre","apellido","edad","sexo"),
  stringdist.match = c("nombre","apellido"),
  numeric.match = c("edad")
)

matches_fastlink <- data.frame(
  idA = fl$matches[,1],
  idB = fl$matches[,2]
)
```

¿Qué hace este bloque?

- Convierte las columnas a tipos adecuados para evitar problemas en la función `fastLink`.
- Ejecuta `fastLink()` con las variables que se usarán para enlazar ambos archivos.
- Indica que `nombre` y `apellido` son campos de texto y que `edad` debe tratarse como variable numérica.
- Extrae los pares emparejados por el algoritmo y los guarda en `matches_fastlink`.

En el contexto del script, este bloque permite contrastar dos enfoques:

1. un método ya implementado en un paquete especializado;
2. una implementación explícita del modelo bayesiano en STAN.

7. Especificación del modelo bayesiano en STAN

El núcleo conceptual del script aparece cuando se define el modelo probabilístico en lenguaje STAN. Este bloque traduce la idea del modelo de Fellegi-Sunter a una formulación que puede estimarse por MCMC.

```
############################################################
# 7. Modelo Fellegi-Sunter bayesiano en Stan
############################################################

stan_code <- "
data {

  int<lower=1> N;               // numero de pares
  int<lower=1> K;               // numero de variables de comparacion
  int<lower=0,upper=1> X[N,K];  // matriz de acuerdos

}

parameters {

  vector<lower=0,upper=1>[K] m;   // probabilidad de acuerdo dado match
  vector<lower=0,upper=1>[K] u;   // probabilidad de acuerdo dado no-match
  real<lower=0,upper=1> pi;       // proporcion de matches

}

model {

  real log_m;
  real log_u;

  m ~ beta(8,2);
  u ~ beta(2,8);
  pi ~ beta(1,5);

  for(i in 1:N){

    log_m = 0;
    log_u = 0;

    for(k in 1:K){

      log_m += bernoulli_lpmf(X[i,k] | m[k]);
      log_u += bernoulli_lpmf(X[i,k] | u[k]);

    }

    target += log_sum_exp(
                log(pi) + log_m,
                log1m(pi) + log_u
```

```
                );

  }

}

"
```

¿Qué hace este bloque?

a) Datos de entrada En el bloque `data` se declaran:

- `N`: número total de pares;
- `K`: número de variables de comparación;
- `X[N,K]`: matriz binaria de acuerdos.

b) Parámetros del modelo En `parameters` aparecen los parámetros clásicos del modelo de Fellegi-Sunter:

- `m[k]`: probabilidad de acuerdo en la variable `k` cuando el par sí es un *match*;
- `u[k]`: probabilidad de acuerdo en la variable `k` cuando el par no es un *match*;
- `pi`: proporción global de pares que son *matches*.

c) Distribuciones previas El modelo define:

- `m ~ beta(8,2)`, prior que favorece acuerdos altos en verdaderos *matches*;
- `u ~ beta(2,8)`, prior que favorece acuerdos bajos en *non-matches*;
- `pi ~ beta(1,5)`, prior que refleja que los *matches* suelen ser poco frecuentes entre todos los pares posibles.

d) Verosimilitud marginal Para cada par `i`, el código calcula:

- `log_m`: log-probabilidad del patrón observado si el par fuera un *match*;
- `log_u`: log-probabilidad del patrón observado si el par fuera un *non-match*.

Luego suma a `target` la contribución marginal del par usando `log_sum_exp`, que implementa de forma estable la expresión:

$$\log\left[\pi \cdot P(X_i \mid Z_i = 1) + (1-\pi) \cdot P(X_i \mid Z_i = 0)\right].$$

Es decir, STAN no observa directamente una etiqueta de *match*, sino que integra sobre esa variable latente.

8. Ajuste del modelo

Una vez especificado el modelo, el script prepara los datos en una lista y llama a STAN para estimar la distribución posterior de los parámetros.

```r
##############################################################
# 8. Ajustar modelo
##############################################################

data_list <- list(
  N = N,
  K = K,
  X = X
)

fit <- stan(
  model_code = stan_code,
  data = data_list,
  iter = 2000,
  chains = 4,
  seed = 123
)
```

¿Qué hace este bloque?

- `data_list` empaqueta la información que STAN espera recibir.
- `stan()` compila el modelo y genera simulaciones MCMC.
- `iter = 2000` indica el número de iteraciones por cadena.
- `chains = 4` ejecuta cuatro cadenas independientes, útil para evaluar convergencia.
- `seed = 123` vuelve reproducible el ajuste.

El resultado `fit` contiene muestras posteriores de `m`, `u` y `pi`.

9. Cálculo de la probabilidad posterior de *match*

Después del ajuste, el script extrae los parámetros estimados y calcula, para cada par, una probabilidad posterior de coincidencia.

```r
##############################################################
# 9. Probabilidad posterior de match
##############################################################

post <- rstan::extract(fit)

m_hat <- colMeans(post$m)
u_hat <- colMeans(post$u)
pi_hat <- mean(post$pi)

p_match <- rep(0,N)

for(i in 1:N){

  log_m <- log(pi_hat)
  log_u <- log(1-pi_hat)

  for(k in 1:K){
```

```
    log_m <- log_m + dbinom(X[i,k],1,m_hat[k],log=TRUE)
    log_u <- log_u + dbinom(X[i,k],1,u_hat[k],log=TRUE)

  }

  p_match[i] <- exp(log_m)/(exp(log_m)+exp(log_u))

}

pairs$p_match <- p_match
pairs$bayes_match <- ifelse(pairs$p_match > 0.5,1,0)

matches_bayes <- pairs[pairs$bayes_match==1,c("idA","idB")]
```

¿Qué hace este bloque?

- `rstan::extract(fit)` recupera las simulaciones posteriores.
- `m_hat`, `u_hat` y `pi_hat` se calculan como resúmenes puntuales de la posterior, usando promedios.
- Para cada par `i`, el código vuelve a calcular:
  - la evidencia a favor de que sea *match*;
  - la evidencia a favor de que sea *non-match*.
- Luego transforma esas cantidades en una probabilidad posterior:

$$P(Z_i = 1 \mid X_i) = \frac{P(Z_i = 1, X_i)}{P(Z_i = 1, X_i) + P(Z_i = 0, X_i)}.$$

- Esa probabilidad se guarda en `pairs$p_match`.
- Después se aplica una regla simple de decisión: si `p_match > 0.5`, el par se clasifica como coincidencia bayesiana.
- Los pares clasificados como coincidencias se almacenan en `matches_bayes`.

Obsérvese que aquí se usa una decisión por umbral fijo de 0.5. En una aplicación real, ese umbral podría modificarse dependiendo del costo de falsos positivos y falsos negativos.

10. Comparación entre métodos

El script termina juntando los resultados de `fastLink` y del procedimiento bayesiano para compararlos en una misma tabla.

```
#############################################################
# 10. Comparar métodos
#############################################################

matches_fastlink$fastlink <- 1
matches_bayes$bayes <- 1

comparison <- merge(
  matches_fastlink,
  matches_bayes,
```

```
  by=c("idA","idB"),
  all=TRUE
)

comparison$fastlink[is.na(comparison$fastlink)] <- 0
```

¿Qué hace este bloque?

- Añade una marca a los pares encontrados por cada método.
- Une ambas salidas con `merge(..., all = TRUE)` para conservar tanto coincidencias compartidas como exclusivas de cada procedimiento.
- Reemplaza los valores faltantes de `fastlink` por 0 cuando un par no fue detectado por ese método.

Aunque el script mostrado termina aquí, el siguiente paso natural sería completar también la columna `bayes` con ceros cuando falte y luego calcular medidas de desempeño, por ejemplo:

- número total de coincidencias encontradas por cada método;
- coincidencias en común;
- precisión y sensibilidad respecto de `true_match`.

11. Lectura global del flujo del script

En conjunto, `RecordLinkage.r` sigue esta secuencia lógica:

1. limpia el entorno y carga librerías;
2. simula dos archivos con coincidencias verdaderas conocidas;
3. genera todos los pares posibles;
4. construye variables binarias de comparación;
5. guarda la verdad simulada para evaluación;
6. ejecuta `fastLink` como referencia;
7. especifica un modelo bayesiano de Fellegi-Sunter en STAN;
8. ajusta el modelo vía MCMC;
9. calcula probabilidades posteriores de *match*;
10. compara los resultados obtenidos por ambos métodos.

De esta manera, el archivo no sólo implementa un ejemplo práctico de *record linkage*, sino que también conecta claramente tres niveles de análisis:

- la simulación de datos;
- la formulación probabilística del problema;
- la implementación computacional de la inferencia bayesiana.